

CSC 139 - Multithreaded Web Server Simulation

Austin Melendez

11/6/2025

Project Introduction

This project we practice the use of locks, conditional variables, semaphores and the consumer/producer/shared buffer problem.

To solve this, we have develop a simulation for a multi-threaded web server in C.

Requirements:

- Https requests are produced to a shared resource (server queue). Producer Thread.
- Requests are consumed from the shared resource. Consumer Thread.
- Have a loop that sends requests and a loop that consumes requests with some kind of delay.
- Implement threads synchronization using:
 - Mutex/locks (pthread).
 - Conditional variables.
 - Semaphores.
- Submit the code with comments, results with screenshot of running the code.
- Submit the process to compile and run your code for testing.
- Explain the differences between your three synchronization implementations (describe what you did) and any challenges you encountered, observations, what worked and what did not work. Which synchronization implementation would you use and why?

Compiling the Project

Once you have extracted the contents of the `webserver.zip`, ensure all 6 files are located.

- `client_server_condvar.c`
- `client_server_mutex.c`
- `client_server_semaphore.c`
- `main.c`
- `server.h`
- `CONCLUSION.md`

Navigate to the directory and run the `gcc` command to compile the projecct.

```
gcc -o webserver main.c client_server_mutex.c client_server_condvar.c  
client_server_semaphore.c -lpthread
```

To view the help menu, run the command `./webserver` in the console:

```
Multi-threaded Web Server Simulation
Usage: ./webserver [mode]
  mode = 1 : Mutex Locks
  mode = 2 : Conditional Variables
  mode = 3 : Semaphores

Example:
  ./webserver 3  # Run with semaphore synchronization

austin_csus@cloudshell:~/labs/webserver$
```

Mutex Synchronization

To run the project with the mutex synchronization method, run the command: `./webserver 1`

Running this command will give you the following output:

```
Multi-threaded Web Server Simulation
Mutex Locks

[Client 1] Sending request: Client 1 -> Request 0
[Client 2] Sending request: Client 2 -> Request 0
[Client 3] Sending request: Client 3 -> Request 0
[Server] Recieved request: Client 1 -> Request 0
[Client 1] Sending request: Client 1 -> Request 1
[Client 2] Sending request: Client 2 -> Request 1
[Client 3] Sending request: Client 3 -> Request 1
[Server] Sending response: Processed response for Client 1 -> Request 0
[Server] Recieved request: Client 2 -> Request 0
[Client 1] Sending request: Client 1 -> Request 2
[Client 2] Sending request: Client 2 -> Request 2
[Client 3] Sending request: Client 3 -> Request 2
[Client 1] Sending request: Client 1 -> Request 3
[Client 2] Sending request: Client 2 -> Request 3
[Client 3] Sending request: Client 3 -> Request 3
[Server] Sending response: Processed response for Client 2 -> Request 0
[Server] Recieved request: Client 3 -> Request 0
[Client 1] Sending request: Client 1 -> Request 4
[Client 2] Sending request: Client 2 -> Request 4
[Client 3] Sending request: Client 3 -> Request 4
[Server] Sending response: Processed response for Client 3 -> Request 0
[Server] Recieved request: Client 1 -> Request 1
[Client 1] Received response: Processed response for Client 1 -> Request 0
[Client 1] Received response: Processed response for Client 2 -> Request 0
[Client 2] Received response: Processed response for Client 3 -> Request 0
[Server] Sending response: Processed response for Client 1 -> Request 1
[Server] Recieved request: Client 2 -> Request 1
[Client 1] Received response: Processed response for Client 1 -> Request 1
[Server] Sending response: Processed response for Client 2 -> Request 1
[Server] Recieved request: Client 3 -> Request 1
[Client 1] Received response: Processed response for Client 2 -> Request 1
[Server] Sending response: Processed response for Client 3 -> Request 1
[Server] Recieved request: Client 1 -> Request 2
[Client 1] Received response: Processed response for Client 3 -> Request 1
[Server] Sending response: Processed response for Client 1 -> Request 2
[Server] Recieved request: Client 2 -> Request 2
[Client 3] Received response: Processed response for Client 1 -> Request 2
[Server] Sending response: Processed response for Client 2 -> Request 2
[Server] Recieved request: Client 3 -> Request 2
[Client 3] Received response: Processed response for Client 2 -> Request 2
[Server] Sending response: Processed response for Client 3 -> Request 2
[Server] Recieved request: Client 1 -> Request 3
[Client 3] Received response: Processed response for Client 3 -> Request 2
[Server] Sending response: Processed response for Client 1 -> Request 3
[Server] Recieved request: Client 2 -> Request 3
[Client 2] Received response: Processed response for Client 1 -> Request 3
[Server] Sending response: Processed response for Client 2 -> Request 3
[Server] Recieved request: Client 3 -> Request 3
[Client 2] Received response: Processed response for Client 2 -> Request 3
[Server] Sending response: Processed response for Client 3 -> Request 3
[Server] Recieved request: Client 1 -> Request 4
[Client 2] Received response: Processed response for Client 3 -> Request 3
[Server] Sending response: Processed response for Client 1 -> Request 4
[Server] Recieved request: Client 2 -> Request 4
[Client 2] Received response: Processed response for Client 1 -> Request 4
[Server] Sending response: Processed response for Client 2 -> Request 4
[Server] Recieved request: Client 3 -> Request 4
[Client 3] Received response: Processed response for Client 2 -> Request 4
[Server] Sending response: Processed response for Client 3 -> Request 4
[Client 3] Received response: Processed response for Client 3 -> Request 4

Simulation Completed!
austin_csus@cloudshell:~/labs/webserver$
```

For mutex synchronization, we used `pthread_mutex_init()` to create a lock that we could unlock and lock using the respective commands `pthread_mutex_lock()` and `pthread_mutex_unlock()`. Using the mutex locks ensures mutual exclusivity when accessing shared data/resources.

Conditional Variable Synchronization

To run the project with the conditional variable synchronization method, run the command: `./webserver 2`

Running this command will give you the following output:

```
Multi-threaded Web Server Simulation
Conditional Variables

[Client 1] Sending request: Client 1 -> Request 0
[Client 2] Sending request: Client 2 -> Request 0
[Client 3] Sending request: Client 3 -> Request 0
[Server] Recieved request: Client 1 -> Request 0
[Client 2] Sending request: Client 2 -> Request 1
[Client 3] Sending request: Client 3 -> Request 1
[Client 1] Sending request: Client 1 -> Request 1
[Server] Sending response: Processed response for Client 1 -> Request 0
[Server] Recieved request: Client 2 -> Request 0
[Client 2] Sending request: Client 2 -> Request 2
[Client 3] Sending request: Client 3 -> Request 2
[Client 1] Sending request: Client 1 -> Request 2
[Server] Sending response: Processed response for Client 2 -> Request 0
[Server] Recieved request: Client 3 -> Request 0
[Client 2] Sending request: Client 2 -> Request 3
[Client 1] Sending request: Client 1 -> Request 3
[Client 3] Sending request: Client 3 -> Request 3
[Client 2] Sending request: Client 2 -> Request 4
[Client 1] Sending request: Client 1 -> Request 4
[Client 3] Sending request: Client 3 -> Request 4
[Server] Sending response: Processed response for Client 3 -> Request 0
[Server] Recieved request: Client 2 -> Request 1
[Client 2] Received response: Processed response for Client 1 -> Request 0
[Client 1] Received response: Processed response for Client 2 -> Request 0
[Client 2] Received response: Processed response for Client 3 -> Request 0
[Server] Sending response: Processed response for Client 2 -> Request 1
[Server] Recieved request: Client 3 -> Request 1
[Client 1] Received response: Processed response for Client 2 -> Request 1
[Server] Sending response: Processed response for Client 3 -> Request 1
[Server] Recieved request: Client 1 -> Request 1
[Client 2] Received response: Processed response for Client 3 -> Request 1
[Server] Sending response: Processed response for Client 1 -> Request 1
[Server] Recieved request: Client 2 -> Request 2
[Client 1] Received response: Processed response for Client 1 -> Request 1
[Server] Sending response: Processed response for Client 2 -> Request 2
[Server] Recieved request: Client 3 -> Request 2
[Client 3] Received response: Processed response for Client 2 -> Request 2
[Server] Sending response: Processed response for Client 3 -> Request 2
[Server] Recieved request: Client 1 -> Request 2
[Client 2] Received response: Processed response for Client 3 -> Request 2
[Server] Sending response: Processed response for Client 1 -> Request 2
[Server] Recieved request: Client 2 -> Request 3
[Client 1] Received response: Processed response for Client 1 -> Request 2
[Server] Sending response: Processed response for Client 2 -> Request 3
[Server] Recieved request: Client 1 -> Request 3
[Client 3] Received response: Processed response for Client 2 -> Request 3
[Server] Sending response: Processed response for Client 1 -> Request 3
[Server] Recieved request: Client 3 -> Request 3
[Client 2] Received response: Processed response for Client 1 -> Request 3
[Server] Sending response: Processed response for Client 3 -> Request 3
[Server] Recieved request: Client 2 -> Request 4
[Client 1] Received response: Processed response for Client 3 -> Request 3
[Server] Sending response: Processed response for Client 2 -> Request 4
[Server] Recieved request: Client 1 -> Request 4
[Client 3] Received response: Processed response for Client 2 -> Request 4
[Server] Sending response: Processed response for Client 1 -> Request 4
[Server] Recieved request: Client 3 -> Request 4
[Client 3] Received response: Processed response for Client 3 -> Request 4

Simulation Completed!
austin_csus@cloudshell:~/labs/webserver$
```

For conditional variable synchronization, we still used `pthread_mutex_init()` to create a lock to ensures mutual exclusivity when accessing shared data/resources. But we also defined two signals using `pthread_cond_init()`. We use these to signal a process to wait if the queue is full or empty using `pthread_cond_wait()` and signal a process to continue if the queue is not full and has at least one element using `pthread_cond_signal()`. Similarly to the mutex solution, we use locks and conditional variables to ensure absolute mutual exclusivity when accessing shared data/resources.

Semaphore Synchronization

To run the project with the semaphore synchronization method, run the command: `./webserver 3`

Running this command will give you the following output:

```
Multi-threaded Web Server Simulation
Semaphores

[Client 1] Sending request: Client 1 -> Request 0
[Server] Recieved request: Client 1 -> Request 0
[Client 2] Sending request: Client 2 -> Request 0
[Client 3] Sending request: Client 3 -> Request 0
[Client 1] Sending request: Client 1 -> Request 1
[Client 3] Sending request: Client 3 -> Request 1
[Client 2] Sending request: Client 2 -> Request 1
[Server] Sending response: Processed response for Client 1 -> Request 0
[Server] Recieved request: Client 2 -> Request 0
[Client 3] Sending request: Client 3 -> Request 2
[Client 2] Sending request: Client 2 -> Request 2
[Client 1] Sending request: Client 1 -> Request 2
[Server] Sending response: Processed response for Client 2 -> Request 0
[Server] Recieved request: Client 3 -> Request 0
[Client 3] Sending request: Client 3 -> Request 3
[Client 2] Sending request: Client 2 -> Request 3
[Client 1] Sending request: Client 1 -> Request 3
[Client 1] Sending request: Client 1 -> Request 4
[Client 3] Sending request: Client 3 -> Request 4
[Client 2] Sending request: Client 2 -> Request 4
[Server] Sending response: Processed response for Client 3 -> Request 0
[Server] Recieved request: Client 1 -> Request 1
[Client 1] Received response: Processed response for Client 1 -> Request 0
[Client 3] Received response: Processed response for Client 2 -> Request 0
[Client 1] Received response: Processed response for Client 3 -> Request 0
[Server] Sending response: Processed response for Client 1 -> Request 1
[Server] Recieved request: Client 3 -> Request 1
[Client 3] Received response: Processed response for Client 1 -> Request 1
[Server] Sending response: Processed response for Client 3 -> Request 1
[Server] Recieved request: Client 2 -> Request 1
[Client 1] Received response: Processed response for Client 3 -> Request 1
[Server] Sending response: Processed response for Client 2 -> Request 1
[Server] Recieved request: Client 3 -> Request 2
[Client 3] Received response: Processed response for Client 2 -> Request 1
[Server] Sending response: Processed response for Client 3 -> Request 2
[Server] Recieved request: Client 2 -> Request 2
[Client 2] Received response: Processed response for Client 3 -> Request 2
[Server] Sending response: Processed response for Client 2 -> Request 2
[Server] Recieved request: Client 1 -> Request 2
[Client 1] Received response: Processed response for Client 2 -> Request 2
[Server] Sending response: Processed response for Client 1 -> Request 2
[Server] Recieved request: Client 3 -> Request 3
[Client 3] Received response: Processed response for Client 1 -> Request 2
[Server] Sending response: Processed response for Client 3 -> Request 3
[Server] Recieved request: Client 2 -> Request 3
[Client 2] Received response: Processed response for Client 3 -> Request 3
[Server] Sending response: Processed response for Client 2 -> Request 3
[Server] Recieved request: Client 1 -> Request 3
[Client 1] Received response: Processed response for Client 2 -> Request 3
[Server] Sending response: Processed response for Client 1 -> Request 3
[Server] Recieved request: Client 1 -> Request 4
[Client 3] Received response: Processed response for Client 1 -> Request 3
[Server] Sending response: Processed response for Client 1 -> Request 4
[Server] Recieved request: Client 3 -> Request 4
[Client 2] Received response: Processed response for Client 1 -> Request 4
[Server] Sending response: Processed response for Client 3 -> Request 4
[Server] Recieved request: Client 2 -> Request 4
[Client 2] Received response: Processed response for Client 3 -> Request 4
[Server] Sending response: Processed response for Client 2 -> Request 4
[Client 2] Received response: Processed response for Client 2 -> Request 4

Simluation Completed!
austin_csus@cloudshell:~/labs/webserver$
```

For semaphores synchronization, we still used `pthread_mutex_init()` to create a lock to ensures mutual exclusivity when accessing shared data/resources. But we also defined semaphores using `sem_init()` based on slots available in the queue and how many items are ready to consume. At the start of all of our producers/consumers we run `sem_wait()` to either wait until there is an empty slot or wait until there is an available item.

Testing

The data that is written by the producer thread and read by the consumer thread is located on line 78 and 118. The request string `Client ID -> Request #` is written to the queue and simulates the https request.

When a client thread sends a request to the server it prints the message: `[Client ID] Sending request: [REQUEST STRING]`

When the server thread reads a request from a client thread it prints the message: `[Server] Recieved request: [REQUEST STRING]`

The response string `Processed response for: Client ID -> Request #` is written to the queue and simulates the https response.

When a client thread reads a response from the server it prints the message: `[Client ID] Received response: [RESPONSE STRING]`

When the server thread sends a response to a client thread it prints the message: `[Server] Sending response: [RESPONSE STRING]`

All of the responses can be seen in figure 1, 2 and 3.

Conclusion

This simulation is set up to mirror a multi-threaded web server where multiple clients (producers) send requests and one server (consumer) processes them.

To safely share the request and response queues between threads, we implemented three synchronization mechanisms:

- mutex locks
- condition variables
- semaphores

Mutex Locks

- Each queue operation (`enqueue_mutex()` and `dequeue_mutex()`) locks the queue before modifying it and unlocks it afterward.
- When the queue is full, the producer (client) repeatedly checks and waits.
- When the queue is empty, the consumer (server) repeatedly checks and waits.
- Used `pthread_mutex_lock()` and `pthread_mutex_unlock()` to protect the shared buffer.

```
pthread_mutex_lock(&q->lock);  
// Check if queue is full or empty, then enqueue/dequeue  
pthread_mutex_unlock(&q->lock);
```

This can cause busy waiting (threads repeatedly check the condition), wasting CPU time when the buffer is full/empty.

- To fix this we must manually add delays (`usleep`) to prevent 100% CPU usage.
- Without condition signaling, threads aren't efficient, they just loop repeatedly.

Conditional Variables

- Added condition variables to signal when the queue becomes non-empty or non-full.
- Used `pthread_cond_wait()` and `pthread_cond_signal()` along with a mutex to coordinate producer/consumer access.
- A producer thread waits on the "not full" condition when the queue is full.
- A consumer thread waits on the "not empty" condition when the queue is empty.
- When items are added or removed, the corresponding condition variable is signaled.

```
pthread_mutex_lock(&q->lock);
while (q->count == MAX_QUEUE)
    pthread_cond_wait(&q->not_full, &q->lock);

// Add item to queue

pthread_cond_signal(&q->not_empty);
pthread_mutex_unlock(&q->lock);
```

Much more efficient than simple mutex locks because threads sleep when waiting.

- Eliminates the need for manual `usleep()` delays or polling.
- Threads are only woken when a condition changes.

Semaphores

- Used two semaphores for each queue:
 - empty: counts how many slots are available for producers.
 - full: counts how many items are available for consumers.
- The producer waits (`sem_wait`) on empty before producing, and signals (`sem_post`) full after adding an item.
- The consumer waits (`sem_wait`) on full before consuming, and signals (`sem_post`) empty after removing an item.
- Also used a mutex to protect internal queue state updates.

```
sem_wait(&q->empty);
pthread_mutex_lock(&q->lock);

// Enqueue item

pthread_mutex_unlock(&q->lock);
sem_post(&q->full);
```

Very efficient, widely used for producer-consumer problems.

- Handles waiting and signaling automatically.
- Semaphores can coordinate multiple producers and consumers easily.

I would choose the Semaphore implementation for a real multi-threaded web server. Reasons:

- Threads block efficiently without consuming CPU.
- Supports multiple concurrent producers and consumers.
- Clear signaling that naturally models the idea of "available slots" and "available requests."
- Performs best when many clients are sending multiple requests at once.